Phase 1 — Yield Optimization AI Demo Starter Project

This is the first working version of the project.

In this phase we will build:

- Project setup
- Folder structure
- Virtual environment setup
- Synthetic manufacturing data generator
- Streamlit dashboard starter
- KPI cards
- Charts
- Industrial UI theme

STEP 1 — Create Project Folder

```
mkdir yield-optimization-ai
cd yield-optimization-ai
```

STEP 2 — Create Virtual Environment

Mac/Linux

```
python3 -m venv venv
source venv/bin/activate
```

Windows

```
python -m venv venv
venv\Scripts\activate
```

STEP 3 — Install Dependencies

Create:

requirements.txt

```
streamlit
pandas
numpy
plotly
scikit-learn
faker
joblib
matplotlib
```

Install:

```
pip install -r requirements.txt
```

STEP 4 — Create Folder Structure

```
yield-optimization-ai/
│
├── app.py
├── requirements.txt
├── README.md
├── .gitignore
│
├── data/
│   └── manufacturing_data.csv
│
├── simulator/
│   └── generate_data.py
│
├── models/
│   ├── train_model.py
│   └── recommendation_engine.py
│
├── pages/
│   ├── 1_Overview.py
```

```
│   ├── 2_Process_Monitoring.py
│   ├── 3_Benchmarking.py
│   └── 4_AI_Recommendations.py
│
├── utils/
│   └── helpers.py
│
└── assets/
    └── style.css
```

STEP 5 — Create Data Generator

simulator/generate_data.py

```
import pandas as pd
import numpy as np
from faker import Faker
from datetime import datetime, timedelta
import random

fake = Faker()

NUM_RECORDS = 5000

plants = ["Plant_A", "Plant_B", "Plant_C"]
lines = ["Line_1", "Line_2", "Line_3"]
shifts = ["Morning", "Evening", "Night"]

records = []

start_date = datetime.now() - timedelta(days=90)

for i in range(NUM_RECORDS):

    timestamp = start_date + timedelta(minutes=i * 30)

    plant = random.choice(plants)
    line = random.choice(lines)
    shift = random.choice(shifts)

    temperature = round(np.random.normal(180, 8), 2)
    pressure = round(np.random.normal(35, 3), 2)
    humidity = round(np.random.normal(55, 10), 2)
```

```
    machine_speed = round(np.random.normal(120, 15), 2)
    vibration = round(np.random.normal(4, 1.2), 2)
    raw_material_quality = round(np.random.uniform(70, 100), 2)
    energy_consumption = round(np.random.normal(450, 40), 2)
    downtime_minutes = max(0, round(np.random.normal(12, 5), 2))

    noise = np.random.normal(0, 2)

    yield_percentage = (
        82
        + (raw_material_quality * 0.12)
        - (humidity * 0.08)
        + (temperature * 0.03)
        - (vibration * 1.5)
        - (downtime_minutes * 0.2)
        + noise
    )

    yield_percentage = round(max(65, min(yield_percentage, 99)), 2)

    scrap_rate = round(100 - yield_percentage + np.random.uniform(0, 3), 2)

    quality_score = round(
        (yield_percentage * 0.7)
        + (raw_material_quality * 0.3),
        2
    )

    predicted_yield = round(
        yield_percentage + np.random.normal(0, 1),
        2
    )

    recommended_temperature = round(temperature + np.random.uniform(-3, 3), 2)
    recommended_pressure = round(pressure + np.random.uniform(-2, 2), 2)

    adoption_flag = random.choice([0, 1])

    records.append({
        "timestamp": timestamp,
        "plant_id": plant,
        "production_line": line,
        "batch_id": fake.uuid4(),
        "temperature": temperature,
        "pressure": pressure,
        "humidity": humidity,
        "machine_speed": machine_speed,
        "vibration": vibration,
```

```
        "raw_material_quality": raw_material_quality,
        "energy_consumption": energy_consumption,
        "operator_shift": shift,
        "yield_percentage": yield_percentage,
        "scrap_rate": scrap_rate,
        "quality_score": quality_score,
        "downtime_minutes": downtime_minutes,
        "recommended_temperature": recommended_temperature,
        "recommended_pressure": recommended_pressure,
        "adoption_flag": adoption_flag,
        "predicted_yield": predicted_yield
    })


df = pd.DataFrame(records)

output_path = "data/manufacturing_data.csv"

df.to_csv(output_path, index=False)

print("Dataset generated successfully")
print(df.head())
```

STEP 6 — Generate Dataset

Run:

```
python simulator/generate_data.py
```

You should now see:

```
data/manufacturing_data.csv
```

STEP 7 — Create Main Streamlit Dashboard

app.py

```
import streamlit as st
```

```
st.set_page_config(
    page_title="Yield Optimization AI",
    page_icon="🏭",
    layout="wide"
)

st.title("🏭 Industrial Yield Optimization Platform")

st.markdown("""
### AI-Powered Manufacturing Intelligence

Monitor yield, optimize process parameters, reduce scrap, and benchmark plants
using AI-driven insights.
""")

st.info("Use the left sidebar to navigate across dashboard modules.")
```

STEP 8 — Create Overview Dashboard

pages/1_Overview.py

```
import streamlit as st
import pandas as pd
import plotly.express as px

st.set_page_config(layout="wide")

st.title("📊 Executive Overview")

# Load Data

df = pd.read_csv("data/manufacturing_data.csv")

# Sidebar Filters

plant = st.sidebar.selectbox(
    "Select Plant",
    ["All"] + list(df["plant_id"].unique())
)

if plant != "All":
    df = df[df["plant_id"] == plant]

# KPI Calculations
```

```
avg_yield = round(df["yield_percentage"].mean(), 2)
avg_scrap = round(df["scrap_rate"].mean(), 2)
avg_quality = round(df["quality_score"].mean(), 2)
avg_energy = round(df["energy_consumption"].mean(), 2)

# KPI Cards

col1, col2, col3, col4 = st.columns(4)

col1.metric("Average Yield %", avg_yield)
col2.metric("Average Scrap %", avg_scrap)
col3.metric("Quality Score", avg_quality)
col4.metric("Energy Consumption", avg_energy)

st.markdown("---")

# Yield Trend

trend = df.groupby("timestamp")["yield_percentage"].mean().reset_index()

fig = px.line(
    trend,
    x="timestamp",
    y="yield_percentage",
    title="Yield Trend Over Time"
)

st.plotly_chart(fig, use_container_width=True)

# Plant Comparison

plant_df = (
    df.groupby("plant_id")[["yield_percentage", "scrap_rate"]]
    .mean()
    .reset_index()
)

fig2 = px.bar(
    plant_df,
    x="plant_id",
    y="yield_percentage",
    color="scrap_rate",
    title="Plant Benchmarking"
)

st.plotly_chart(fig2, use_container_width=True)
```

STEP 9 — Create Process Monitoring Page

pages/2_Process_Monitoring.py

```
import streamlit as st
import pandas as pd
import plotly.express as px

st.set_page_config(layout="wide")

st.title("⚙️ Process Monitoring")

# Load data

df = pd.read_csv("data/manufacturing_data.csv")

# Sensor Trends

sensor = st.selectbox(
    "Select Sensor",
    [
        "temperature",
        "pressure",
        "humidity",
        "machine_speed",
        "vibration"
    ]
)

fig = px.line(
    df,
    x="timestamp",
    y=sensor,
    color="plant_id",
    title=f"{sensor} Monitoring"
)

st.plotly_chart(fig, use_container_width=True)

# Correlation Analysis

st.subheader("Correlation Analysis")

corr_df = df[[
```

```
    "temperature",
    "pressure",
    "humidity",
    "machine_speed",
    "vibration",
    "yield_percentage",
    "scrap_rate"
]].corr()

st.dataframe(corr_df)
```

STEP 10 — Create Benchmarking Page

pages/3_Benchmarking.py

```
import streamlit as st
import pandas as pd
import plotly.express as px

st.set_page_config(layout="wide")

st.title("🏭 Cross Plant Benchmarking")

# Load Data

df = pd.read_csv("data/manufacturing_data.csv")

benchmark = (
    df.groupby("plant_id")[[
        "yield_percentage",
        "scrap_rate",
        "quality_score",
        "energy_consumption"
    ]]
    .mean()
    .reset_index()
)

metric = st.selectbox(
    "Select KPI",
    [
        "yield_percentage",
        "scrap_rate",
        "quality_score",
```

```
        "energy_consumption"
    ]
)

fig = px.bar(
    benchmark,
    x="plant_id",
    y=metric,
    color=metric,
    title=f"Plant Comparison — {metric}"
)

st.plotly_chart(fig, use_container_width=True)
```

STEP 11 — Create AI Recommendations Page

pages/4_AI_Recommendations.py

```
import streamlit as st
import pandas as pd

st.set_page_config(layout="wide")

st.title("🤖 AI Recommendation Center")

# Load Data

df = pd.read_csv("data/manufacturing_data.csv")

latest = df.tail(20)

for _, row in latest.iterrows():

    recommendation = []

    if row["humidity"] > 60:
        recommendation.append("Reduce humidity below 55%")

    if row["vibration"] > 5:
        recommendation.append("Inspect machine vibration levels")

    if row["temperature"] < 175:
        recommendation.append("Increase temperature by 2-3°C")
```

```
    if len(recommendation) == 0:
        recommendation.append("Process operating optimally")

    with st.expander(f"Batch: {row['batch_id'][:8]}"):

        st.write(f"Plant: {row['plant_id']}")
        st.write(f"Predicted Yield: {row['predicted_yield']}%")
        st.write(f"Current Yield: {row['yield_percentage']}%")

        st.markdown("### Recommendations")

        for rec in recommendation:
            st.success(rec)
```

STEP 12 — Optional Industrial Styling

assets/style.css

```
.main {
    background-color: #0e1117;
}

h1, h2, h3 {
    color: #00d4ff;
}

.stMetric {
    background-color: #1f2937;
    padding: 15px;
    border-radius: 10px;
}
```

STEP 13 — Run Streamlit App

```
streamlit run app.py
```

What You Will Have After This

✅ Industrial AI dashboard

✅ Manufacturing dataset

✅ Yield KPIs

✅ Process monitoring

✅ Plant benchmarking

✅ AI recommendations

✅ Interactive charts

✅ Multi-page navigation

PHASE 2 (NEXT)

Next we will build:

- Real ML model training
- Yield prediction model
- Scrap prediction
- Anomaly detection
- Better UI theme
- Real-time simulation
- Semantic KPI layer
- Download reports
- Digital twin style dashboard
- Advanced charts
- SQL database integration
- Login system
- AI chatbot assistant